UD4. Transaccións e integridade dos datos

Nas sesións anteriores utilizamos sentenzas SQL para inserir, actualizar e eliminar información, así como guións e procedementos almacenados para automatizar operacións. Porén, cando unha tarefa implica varias modificacións relacionadas, é importante garantir que todas se realizan correctamente.

Imaxinemos unha transferencia bancaria entre dúas contas. Para realizar esta operación é necesario restar unha cantidade da conta de orixe e sumala na conta de destino. Se a primeira modificación se realiza correctamente pero a segunda falla, o sistema quedaría nun estado incorrecto: desaparecería diñeiro dunha conta sen aparecer na outra.

As transaccións permiten agrupar varias operacións nunha unidade de traballo. Desta maneira, podemos confirmar todos os cambios cando a operación remata correctamente ou cancelalos se se produce algún problema.

1. Introdución ás transaccións

1.1. Concepto de transacción

Unha transacción é un conxunto de operacións SQL que se executan como unha única unidade lóxica de traballo. O seu obxectivo é garantir que todas as operacións se completen correctamente ou que ningunha delas quede aplicada de maneira parcial.

Durante unha transacción podemos executar diferentes sentenzas SQL, especialmente operacións de manipulación de datos:

- `INSERT` : inserir rexistros.
- `UPDATE` : modificar rexistros existentes.
- `DELETE` : eliminar rexistros.
- `SELECT` : consultar información para comprobar o estado dos datos.

Os cambios realizados durante a transacción poden permanecer pendentes ata que se confirmen mediante `COMMIT` ou se cancelen mediante `ROLLBACK` .

1.2. Finalidade das transaccións

As transaccións utilízanse cando varias operacións deben manter unha relación lóxica entre si. Os seus principais obxectivos son:

- Evitar que unha operación quede executada parcialmente.

- Manter a coherencia e integridade da información.
- Permitir cancelar cambios cando se produce un erro.
- Coordinar operacións realizadas por varios usuarios simultaneamente.
- Garantir que os cambios confirmados permanezan almacenados.

1.3. Exemplo dunha operación sen transacción

Supoñamos que temos unha táboa de contas bancarias:

```
CREATE TABLE contas (
    id INT PRIMARY KEY,
    titular VARCHAR(100) NOT NULL,
    saldo DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;

INSERT INTO contas (id, titular, saldo)
VALUES
    (1, 'Ana García', 1000.00),
    (2, 'Luis López', 500.00);
```

Ana quere transferir 100 euros á conta de Luis. A operación require dúas actualizacións:

```
UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

UPDATE contas
SET saldo = saldo + 100
WHERE id = 2;
```

Se a primeira sentenza se executa correctamente pero a segunda produce un erro, a conta de Ana terá 100 euros menos, mentres que a conta de Luis conservará o saldo anterior.

A información quedaría nun estado incorrecto. Para evitar esta situación debemos executar as dúas actualizacións dentro dunha transacción.

2. Propiedades ACID

2.1. Que significa ACID?

As transaccións deben cumprir catro propiedades fundamentais coñecidas co acrónimo ACID. Estas propiedades describen as garantías que debe ofrecer un sistema de xestión de bases de datos para manter a información consistente e fiable.

- **A**: Atomicidade (Atomicity).
- **C**: Consistencia (Consistency).
- **I**: Illamento (Isolation).
- **D**: Durabilidade (Durability).

2.2. Atomicidade

A atomicidade significa que unha transacción se considera unha unidade indivisible: ou se realizan todas as operacións correctamente ou non se aplica ningunha delas.

No exemplo da transferencia bancaria, a retirada dos cartos da conta de orixe e o ingreso na conta de destino forman parte dunha mesma operación lóxica.

Se falla algunha das dúas modificacións, a transacción debe cancelarse para evitar que quede aplicada só unha parte.

> **Exemplo:** nunha transferencia de 100 euros, non pode quedar aplicada a retirada dos 100 euros sen que se realice tamén o ingreso na conta de destino.

2.3. Consistencia

A consistencia garante que unha transacción leva a base de datos dun estado válido a outro estado válido, respectando as regras e restricións definidas.

Entre estas regras podemos atopar:

- Claves primarias que non admiten valores duplicados.
- Claves foráneas que garanten as relacións entre táboas.
- Restricións `NOT NULL`.
- Restricións `UNIQUE`.
- Restricións `CHECK`.
- Regras de negocio implementadas mediante operacións SQL.

Por exemplo, podemos definir unha restrición para impedir que o saldo dunha conta sexa negativo:

```
CREATE TABLE contas_validas (
    id INT PRIMARY KEY,
    titular VARCHAR(100) NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
    CHECK (saldo >= 0)
) ENGINE=InnoDB;
```

Se unha actualización tenta establecer un saldo negativo, a restrición impedirá a operación.

> **Importante:** a consistencia non significa que MariaDB coñeza automaticamente todas as regras do negocio. As restricións e condicións necesarias deben definirse correctamente mediante o deseño da base de datos e a lóxica das aplicacións ou procedementos.

2.4. Illamento

O illamento garante que as transaccións que se executan simultaneamente non interfiran de maneira incorrecta unhas nas outras.

Nunha base de datos poden existir varios usuarios traballando ao mesmo tempo. Por exemplo:

- Un usuario está actualizando o saldo dunha conta.
- Outro usuario está consultando o saldo desa mesma conta.
- Un terceiro usuario está realizando outra operación sobre a mesma información.

O sistema debe controlar estas operacións para evitar resultados incorrectos derivados da execución simultánea.

O illamento non implica necesariamente que todas as transaccións se executen unha despois doutra. O SXBD pode permitir a súa execución concorrente, aplicando mecanismos de control para manter a coherencia.

2.5. Durabilidade

A durabilidade significa que os cambios confirmados mediante `COMMIT` deben permanecer almacenados, mesmo se posteriormente se produce unha caída do sistema ou un reinicio do servidor.

MariaDB utiliza mecanismos de rexistro e recuperación para intentar garantir que as transaccións confirmadas non se perdan ante fallos. En InnoDB, estes mecanismos inclúen o rexistro de transaccións (*redo log*).

2.6. Resumo das propiedades ACID

Propiedade	Significado	Problema que evita
Atomicidade	Todas as operacións se realizan ou ningunha queda aplicada.	Operacións incompletas.
Consistencia	Respéctanse as regras e restricións da base de datos.	Estados incorrectos dos datos.
Illamento	As transaccións concorrentes non interfiren incorrectamente.	Problemas derivados do acceso simultáneo.
Durabilidade	Os cambios confirmados permanecen almacenados.	Perda de cambios confirmados.

3. Motores de almacenamento e transaccións en MariaDB

3.1. Que é un motor de almacenamento?

MariaDB permite utilizar diferentes motores de almacenamento para xestionar a información das táboas. Cada motor implementa os seus propios mecanismos de almacenamento, acceso, bloqueo e recuperación.

Non todos os motores ofrecen as mesmas funcionalidades. Unha das diferenzas máis importantes é o soporte para transaccións.

3.2. InnoDB

InnoDB é o motor de almacenamento recomendado para a maioría das aplicacións que necesitan garantir a integridade e coherencia dos datos.

Entre as súas características destacan:

- Soporte para transaccións.
- Compatibilidade coas propiedades ACID.
- Soporte para claves foráneas.
- Bloqueo a nivel de fila en operacións habituais.
- Mecanismos de recuperación ante fallos.
- Control da concorrencia mediante mecanismos de bloqueo e versións dos datos.

Para crear unha táboa utilizando InnoDB:

```
CREATE TABLE produtos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    prezo DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;
```

3.3. MyISAM

MyISAM é outro motor de almacenamento de MariaDB, máis antigo, que non ofrece soporte para transaccións.

Entre as súas características destacan:

- Non permite utilizar `COMMIT` e `ROLLBACK` para desfacer cambios nas súas táboas.
- Non ofrece soporte para claves foráneas.
- Utiliza bloqueo a nivel de táboa.
- Non proporciona as garantías transaccionais ACID de InnoDB.

Podemos crear unha táboa utilizando MyISAM do seguinte xeito:

```
CREATE TABLE rexistros_historicos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    descricion VARCHAR(200),
    data_rexistro DATE
) ENGINE=MyISAM;
```

3.4. Comparación entre InnoDB e MyISAM

Característica	InnoDB	MyISAM
Transaccións	Si	Non
ACID	Si	Non
Claves foráneas	Si	Non
Bloqueo habitual	A nivel de fila	A nivel de táboa
Recuperación transaccional	Si	Non
Uso recomendado	Aplicacións con integridade e transaccións.	Táboas que non precisan funcionalidades transaccionais.

> **Conclusión:** para traballar con transaccións debemos utilizar táboas que empreguen un motor transaccional, como InnoDB. Non abonda con executar `START TRANSACTION` : se as táboas utilizan un motor non transaccional, os seus cambios non se poderán desfacer mediante `ROLLBACK` .

3.5. Consultar o motor dunha táboa

Podemos comprobar que motor utiliza unha táboa mediante:

```
SHOW TABLE STATUS
FROM academia
WHERE Name = 'alumnos';
```

Outra opción é consultar a definición da táboa:

```
SHOW CREATE TABLE alumnos;
```

Se necesitamos cambiar o motor dunha táboa existente, podemos utilizar:

```
ALTER TABLE alumnos ENGINE=InnoDB;
```

4. Inicio e finalización de transaccións

4.1. Inicio dunha transacción

Para iniciar unha transacción de maneira explícita en MariaDB utilizamos a sentenza `START TRANSACTION`.

```
START TRANSACTION;
```

A partir dese momento, as operacións transaccionais realizadas na conexión quedan pendentes de confirmación ou cancelación.

4.2. Confirmación mediante COMMIT

A sentenza `COMMIT` confirma todos os cambios realizados na transacción. Unha vez confirmados, os cambios pasan a ser permanentes e xa non poden desfacerse mediante un `ROLLBACK` ordinario.

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

UPDATE contas
SET saldo = saldo + 100
WHERE id = 2;

COMMIT;
```

Neste exemplo, as dúas actualizacións forman parte dunha mesma transacción. Se ambas se executan correctamente, `COMMIT` confirma os cambios.

4.3. Cancelación mediante ROLLBACK

A sentenza `ROLLBACK` cancela os cambios realizados desde o inicio da transacción ou desde o último punto de restauración ao que se volva.

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

UPDATE contas
SET saldo = saldo + 100
WHERE id = 2;

ROLLBACK;
```

Neste caso, as actualizacións quedan desfeitas e os saldos recuperan os valores que tiñan antes de iniciar a transacción.

4.4. Sintaxe básica dunha transacción

A estrutura habitual dunha transacción é:

```
START TRANSACTION;

-- Operacións SQL
INSERT ...;
UPDATE ...;
DELETE ...;

-- Se todo é correcto
COMMIT;

-- Se se produce algún problema
-- ROLLBACK;
```

Na práctica, `COMMIT` e `ROLLBACK` son alternativas: debemos executar unha delas para finalizar a transacción.

> **Atención:** non se debe executar `COMMIT` e despois `ROLLBACK` esperando recuperar os datos anteriores. Unha vez confirmado o cambio, o `ROLLBACK` xa non pode desfacer esa transacción.

5. Confirmación e cancelación de cambios

5.1. Cambios pendentes e cambios permanentes

Durante unha transacción, as modificacións realizadas aínda non foron confirmadas. O seu resultado pode ser revisado pola propia conexión antes de decidir se se confirman ou se cancelan.

A diferenza entre os cambios pendentes e os confirmados é fundamental:

Estado	Significado	Posibilidade de cancelación
Cambios pendentes	Operacións realizadas dentro dunha transacción aínda non confirmada.	Si, mediante `ROLLBACK`.
Cambios confirmados	Operacións finalizadas mediante `COMMIT`.	Non mediante un ROLLBACK ordinario.

5.2. Comprobar os cambios antes de confirmalos

Unha boa práctica consiste en consultar os datos afectados antes de executar `COMMIT`. Isto permite comprobar que as modificacións son correctas.

Exemplo:

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

-- Comprobar o saldo da conta
SELECT *
FROM contas
WHERE id = 1;

-- Se o resultado é correcto:
COMMIT;

-- Se o resultado non é correcto:
-- ROLLBACK;
```

As consultas realizadas dentro da mesma conexión permiten revisar os cambios pendentes antes de confirmalos.

5.3. Autocommit

MariaDB dispón dunha opción chamada `autocommit` que controla se cada sentenza se confirma automaticamente ao finalizar.

Por defecto, normalmente está activada. Isto significa que cada operación individual sobre unha táboa transaccional se confirma automaticamente se non se iniciou unha transacción explícita.

Podemos consultar o seu estado mediante:

```
SELECT @@autocommit;
```

Para desactivar temporalmente o modo de confirmación automática na conexión actual:

```
SET autocommit = 0;
```

Para volver activalo:

```
SET autocommit = 1;
```

Tamén podemos consultar e modificar esta opción mediante a configuración da sesión.

> **Importante:** cambiar `autocommit` non é o mesmo que iniciar unha transacción explícita. Ademais, activar `autocommit` mentres existe unha transacción pendente pode provocar a súa confirmación. Para exercicios e operacións controladas é recomendable utilizar explicitamente `START TRANSACTION`, `COMMIT` e `ROLLBACK`.

5.4. Confirmacións implícitas

Algunhas sentenzas SQL provocan unha confirmación implícita (*implicit commit*) das transaccións. Isto significa que poden finalizar unha transacción pendente independentemente de que teñamos previsto executar posteriormente un `ROLLBACK`.

Entre as operacións que poden provocar esta situación están:

- `CREATE TABLE`.
- `ALTER TABLE`.
- `DROP TABLE`.

- `TRUNCATE TABLE`.
- `CREATE DATABASE`.
- `DROP DATABASE`.

Por este motivo, debemos evitar mesturar operacións DDL e operacións DML dentro dunha mesma transacción cando esperamos poder desfacer todos os cambios mediante `ROLLBACK`.

6. Puntos de restauración con SAVEPOINT

6.1. Concepto de SAVEPOINT

Un punto de restauración permite marcar unha posición dentro dunha transacción para poder regresar a ela posteriormente sen cancelar necesariamente todas as operacións realizadas.

Para crear un punto de restauración utilizamos:

```
SAVEPOINT nome_punto;
```

A partir dese momento, podemos desfacer parcialmente os cambios realizados despois dese punto mediante `ROLLBACK TO SAVEPOINT`.

6.2. Crear puntos de restauración

Imos utilizar unha transacción que modifica varias contas bancarias. Crearemos puntos de restauración para poder desfacer unha parte das operacións.

```
START TRANSACTION;

UPDATE contas
SET saldo = saldo - 100
WHERE id = 1;

SAVEPOINT punto1;

UPDATE contas
SET saldo = saldo + 100
WHERE id = 2;

SAVEPOINT punto2;

UPDATE contas
SET saldo = saldo - 50
WHERE id = 1;
```

Neste momento realizáronse tres actualizacións e existen dous puntos de restauración.

6.3. Recuperación parcial con ROLLBACK TO SAVEPOINT

Se queremos desfacer unicamente as operacións realizadas despois de `punto2`, podemos executar:

```
ROLLBACK TO SAVEPOINT punto2;
```

A actualización que restaba 50 euros da conta 1 queda desfeita. As operacións anteriores a `punto2` permanecen pendentes.

Podemos continuar realizando modificacións ou confirmar a transacción:

```
COMMIT;
```

6.4. Eliminar puntos de restauración

Se xa non necesitamos un punto de restauración, podemos eliminalo mediante `RELEASE SAVEPOINT`.

```
RELEASE SAVEPOINT punto1;
```

A eliminación do punto de restauración non desfai as operacións realizadas. Simplemente elimina a posibilidade de regresar a ese punto.

6.5. Diferenza entre ROLLBACK e ROLLBACK TO SAVEPOINT

Sentenza	Resultado
`ROLLBACK`	Desfai todos os cambios pendentes da transacción.
`ROLLBACK TO SAVEPOINT nome`	Desfai os cambios realizados despois do punto indicado.
`RELEASE SAVEPOINT nome`	Elimina o punto de restauración sen desfacer cambios.

> **Importante:** ao executar `ROLLBACK TO SAVEPOINT`, a transacción continúa aberta. Non se confirma nin se cancela por completo. Ademais, os puntos de restauración posteriores ao punto ao que se regresa deixan de estar dispoñibles.

7. Transaccións con varias operacións DML

7.1. Operacións sobre varias táboas relacionadas

As transaccións son especialmente útiles cando unha operación de negocio require modificar varias táboas relacionadas.

Por exemplo, ao realizar unha compra podemos necesitar:

1. Inserir un novo pedido.
2. Inserir os produtos asociados ao pedido.
3. Actualizar o stock dos produtos.
4. Actualizar información adicional do cliente ou do pedido.

Todas estas operacións deberían realizarse correctamente para que a compra quede rexistrada. Se falla algunha delas, pode ser necesario cancelar o proceso completo.

7.2. Exemplo: rexistrar unha matrícula

Supoñamos que temos as táboas `alumnos`, `asignaturas` e `matriculas`. Queremos matricular un alumno nunha asignatura e establecer unha nota inicial.

A táboa `matriculas` utiliza unha clave primaria composta polos identificadores do alumno e da asignatura.

```
START TRANSACTION;

INSERT INTO matriculas (
    alumno_id,
    asignatura_id,
    nota
)
VALUES (1, 2, NULL);

UPDATE alumnos
SET edad = edad
WHERE id = 1;

COMMIT;
```

O exemplo mostra como agrupar varias operacións nunha mesma transacción. A actualización de idade non modifica o seu valor, polo que se utiliza unicamente como exemplo dunha segunda operación sobre outra táboa. Nunha aplicación real, esa operación debería substituírse por unha modificación necesaria para o proceso.

7.3. Exemplo: operación de compra

Imos considerar unha base de datos simplificada para xestionar produtos e pedidos:

```
CREATE TABLE produtos (
    id INT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    stock INT NOT NULL,
    CHECK (stock >= 0)
) ENGINE=InnoDB;

CREATE TABLE pedidos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    data_pedido DATE NOT NULL,
    importe DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE detalle_pedidos (
    pedido_id INT,
    produto_id INT,
    cantidade INT NOT NULL,
    prezo DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (pedido_id, produto_id),
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id),
    FOREIGN KEY (produto_id) REFERENCES produtos(id)
) ENGINE=InnoDB;
```

Para rexistrar unha compra debemos inserir o pedido, engadir o detalle e actualizar o stock. Estas operacións deben formar parte dunha mesma transacción.

```
START TRANSACTION;

-- 1. Crear o pedido
INSERT INTO pedidos (data_pedido, importe)
VALUES (CURRENT_DATE(), 100.00);

-- 2. Gardar o identificador do pedido
SET @pedido_id = LAST_INSERT_ID();

-- 3. Inserir o detalle
INSERT INTO detalle_pedidos (
    pedido_id,
    produto_id,
    cantidade,
    prezo
)
VALUES (
    @pedido_id,
    1,
    2,
    50.00
);

-- 4. Actualizar o stock
UPDATE produtos
SET stock = stock - 2
WHERE id = 1
  AND stock >= 2;

-- 5. Confirmar a operación
COMMIT;
```

O exemplo ilustra a secuencia de operacións, pero unha implementación real debe comprobar que a actualización do stock afectou exactamente ao número de produtos esperado. Se non hai stock suficiente, non se debe confirmar o pedido.

7.4. Comprobación do resultado dunha operación

Para garantir que o proceso se completou correctamente, podemos consultar o número de filas afectadas mediante `ROW_COUNT()`. Esta función devolve o número de filas afectadas pola última sentenza que modifica datos.

Nunha aplicación ou procedemento almacenado podemos utilizar ese resultado para decidir se debemos confirmar ou cancelar a transacción.

```
START TRANSACTION;

UPDATE produtos
SET stock = stock - 2
WHERE id = 1
  AND stock >= 2;

SELECT ROW_COUNT() AS filas_modificadas;
```

Se a actualización afecta unha fila, significa que se realizou a modificación. Se afecta cero filas, pode significar que o produto non existe ou que non hai stock suficiente.

Nun guión executado de maneira manual, debemos comprobar o resultado antes de decidir se executamos `COMMIT` ou `ROLLBACK`.

8. Exemplo completo: transferencia bancaria segura

8.1. Preparación das táboas

Imos desenvolver un exemplo completo que permita realizar transferencias bancarias utilizando transaccións.

En primeiro lugar, creamos a táboa:

```
CREATE TABLE contas_bancarias (
    id INT PRIMARY KEY,
    titular VARCHAR(100) NOT NULL,
    saldo DECIMAL(10,2) NOT NULL,
    CHECK (saldo >= 0)
) ENGINE=InnoDB;
```

Inserimos dúas contas de exemplo:

```
INSERT INTO contas_bancarias (id, titular, saldo)
VALUES
    (1, 'Ana García', 1000.00),
    (2, 'Luis López', 500.00);
```

Comprobamos os saldos:

```
SELECT * FROM contas_bancarias;
```

8.2. Executar unha transferencia

Agora queremos transferir 200 euros da conta de Ana á conta de Luis. A operación debe comprobar que existe saldo suficiente antes de realizar a transferencia.

```
START TRANSACTION;

-- Consultar o saldo da conta de orixe
SELECT saldo
FROM contas_bancarias
WHERE id = 1;

-- Retirar 200 euros da conta de orixe
UPDATE contas_bancarias
SET saldo = saldo - 200
WHERE id = 1
  AND saldo >= 200;

-- Consultar o resultado da actualización
SELECT ROW_COUNT() AS filas_modificadas;

-- Engadir 200 euros á conta de destino
UPDATE contas_bancarias
SET saldo = saldo + 200
WHERE id = 2;

-- Comprobar os saldos
SELECT * FROM contas_bancarias;

-- Se todo é correcto:
COMMIT;

-- Se se detecta algún problema:
-- ROLLBACK;
```

A transferencia só debe confirmarse se a retirada e o ingreso se realizaron correctamente. Se a conta de orixe non ten saldo suficiente, a operación de retirada non modificará ningún rexistro.

> **Precaución:** o exemplo permite visualizar as operacións e comprobar manualmente o resultado. Nunha aplicación real tamén se debe comprobar que existen ambas as contas e que cada actualización afecta exactamente a unha fila. Ademais, para evitar condicións de concorrencia é recomendable bloquear as filas implicadas durante a operación.

8.3. Probar a cancelación da transferencia

Podemos repetir a operación, pero esta vez cancelala antes de confirmar os cambios:

```
START TRANSACTION;

UPDATE contas_bancarias
SET saldo = saldo - 100
WHERE id = 1;

UPDATE contas_bancarias
SET saldo = saldo + 100
WHERE id = 2;

-- Cancelar todas as modificacións
ROLLBACK;

-- Comprobar que os saldos non cambiaron
SELECT * FROM contas_bancarias;
```

Despois de executar `ROLLBACK`, os saldos deben conservar os valores que tiñan antes de iniciar a transacción.

9. Boas prácticas no uso de transaccións

Para traballar correctamente con transaccións é recomendable seguir estas pautas:

- **Utilizar InnoDB:** comprobar que todas as táboas implicadas utilizan un motor transaccional.
- **Iniciar a transacción cando sexa necesario:** agrupar as operacións que forman parte dunha mesma tarefa.
- **Comprobar os resultados:** verificar que as sentenzas afectaron aos rexistros esperados.
- **Confirmar só cando todo sexa correcto:** executar `COMMIT` unha vez completadas e comprobadas as operacións.
- **Cancelar ante un erro:** utilizar `ROLLBACK` se algunha operación non pode completarse.

- **Utilizar SAVEPOINT:** crear puntos de restauración cando se necesite desfacer parcialmente unha transacción.
- **Evitar transaccións demasiado longas:** mantelas abertas unicamente durante o tempo necesario.
- **Ter en conta a concorrencia:** considerar que outros usuarios poden estar modificando os mesmos datos.
- **Evitar operacións DDL:** non mesturar creacións ou modificacións de estruturas con operacións que se espera poder desfacer mediante `ROLLBACK`.

10. Resumo das principais sentenzas

Sentenza	Finalidade
`START TRANSACTION`	Iniciar unha nova transacción.
`COMMIT`	Confirmar os cambios realizados.
`ROLLBACK`	Cancelar os cambios pendentes.
`SAVEPOINT nome`	Crear un punto de restauración.
`ROLLBACK TO SAVEPOINT nome`	Desfacer os cambios posteriores a un punto.
`RELEASE SAVEPOINT nome`	Eliminar un punto de restauración.
`SET autocommit = 0`	Desactivar a confirmación automática na sesión.
`SET autocommit = 1`	Activar a confirmación automática na sesión.

11. Guión de comprobación

Antes de finalizar unha operación que utiliza transaccións, convén revisar os seguintes aspectos:

1. Comprobar que as táboas implicadas utilizan InnoDB.
2. Identificar todas as operacións que forman parte da tarefa.
3. Iniciar a transacción mediante `START TRANSACTION`.
4. Executar as sentenzas DML necesarias.
5. Comprobar que cada operación afecta aos rexistros esperados.
6. Verificar que se manteñen as restricións de integridade.

7. Se todo é correcto, confirmar mediante `COMMIT`.
8. Se se produce algún erro, cancelar mediante `ROLLBACK`.
9. Se só é necesario desfacer unha parte, utilizar `SAVEPOINT`.
10. Consultar os datos finais para comprobar o resultado.

> **Conclusión:** as transaccións permiten executar varias operacións como unha única unidade de traballo, mantendo a integridade e coherencia da base de datos. As propiedades ACID definen as garantías fundamentais deste mecanismo, mentres que `COMMIT`, `ROLLBACK` e `SAVEPOINT` permiten controlar a confirmación e recuperación dos cambios.